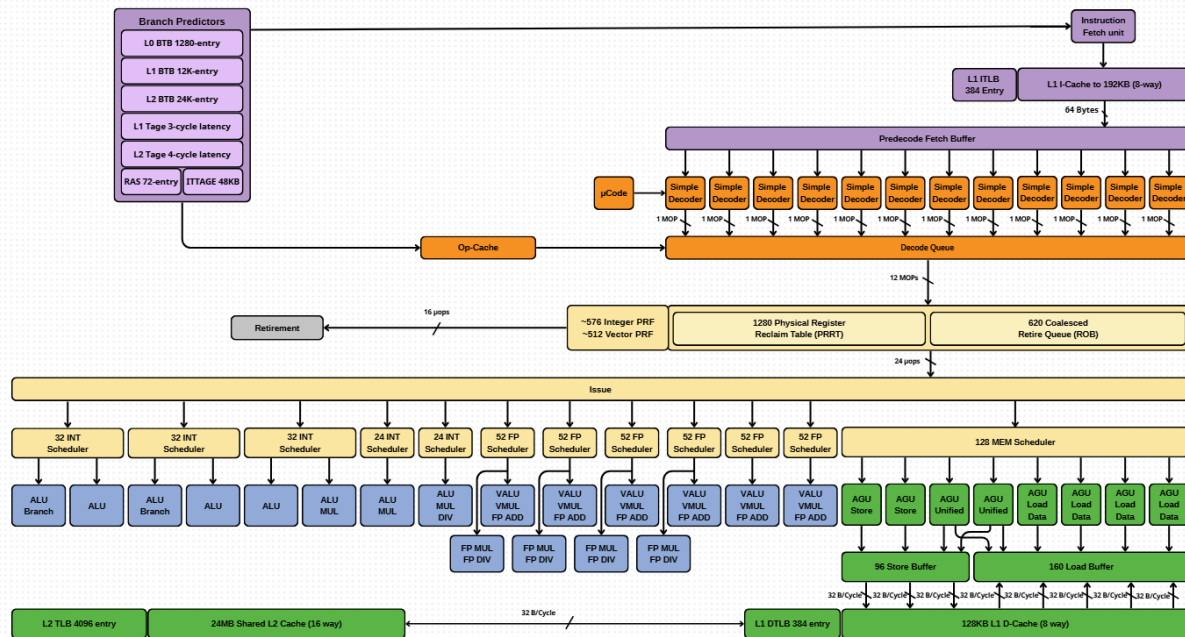


MicroArchitecture

P core

P Core



Phân xưởng 1: Front-end (Bộ phận Tìm kiếm & Phiên dịch)

Mục tiêu của bộ phận này là luôn giữ cho nhà máy có việc để làm, không bao giờ được thiếu nguyên liệu (lệnh).

1. Khối Branch Predictors (Dự đoán rẽ nhánh - Cột màu tím bên trái)

Khi phần mềm chạy, nó không đi theo một đường thẳng mà liên tục có các ngã rẽ (ví dụ: Nếu $A > B$ thì làm X, ngược lại làm Y). CPU không thể chờ tính xong A và B rồi mới nạp lệnh X hoặc Y vì như vậy sẽ rất chậm. Nó phải "đoán" trước.

- **L0 / L1 / L2 BTB (Branch Target Buffer):** Đây là các bộ nhớ đệm lưu lại "địa chỉ đích" của các lần rẽ nhánh trước đó. Sơ đồ này có tới 3 tầng cấp bậc: L0 siêu nhanh (1280 mục), L1 (12 ngàn mục) và L2 khổng lồ (24 ngàn mục). Khi gặp ngã rẽ, CPU lục tìm trong đây xem đích đến tiếp theo có thể là gì.
- **L1/L2 TAGE:** TAGE là thuật toán dự đoán rẽ nhánh tiên tiến nhất hiện nay. Nó phân tích lịch sử các lần rẽ nhánh trước đó để tìm ra quy luật với độ chính xác cực cao. Độ trễ (latency) của L1 là 3 chu kỳ, L2 là 4 chu kỳ.
- **RAS (Return Address Stack - 72 entry):** Chuyên dùng để dự đoán khi một hàm (function) kết thúc thì nó sẽ quay về dòng code nào.
- **ITTAGE (Indirect Target TAGE - 48KB):** Dự đoán các điểm rẽ nhánh gián tiếp (địa chỉ đích thay đổi liên tục).

2. Khối Instruction Fetch (Nạp lệnh)

- Nhận kết quả dự đoán, **Instruction Fetch Unit** sẽ đi lấy lệnh từ RAM.
- **L1 I-Cache (192KB - 8 way)**: Đây là kho chứa lệnh tạm thời siêu tốc. Mức dung lượng 192KB là **cực kỳ lớn** (các CPU thông thường chỉ có 32KB hoặc 64KB). Nó cho phép kéo một lượng dữ liệu khổng lồ **64 Bytes mỗi chu kỳ** nhịp đồng hồ.
- **L1 ITLB (Instruction Translation Lookaside Buffer - 384 entry)**: Khi phần mềm yêu cầu địa chỉ ảo, khối này dịch ngay lập tức sang địa chỉ vật lý của bộ nhớ để đi lấy lệnh.

3. Khối Decode (Giải mã lệnh - Màu cam)

Lệnh kéo về là ngôn ngữ của phần mềm (như x86 hay ARM), nhưng máy móc bên trong chỉ hiểu các vi lệnh nhỏ xíu gọi là MOPs/ μ Sops (Micro-ops).

- **Predecode Fetch Buffer**: Bộ đệm chuẩn bị trước khi giải mã.
- **12 Simple Decoders**: Đây là điểm nhấn kinh hoàng của kiến trúc này. Nó có tới **12 bộ giải mã** hoạt động song song. Hầu hết các CPU tối tân hiện nay chỉ giải mã 4 đến 8 lệnh/chu kỳ. 12 bộ giải mã giúp CPU "nhai" được một lượng lệnh khổng lồ cùng lúc.
- **μ Code**: Các lệnh quá phức tạp (ví dụ lệnh chia, lệnh chép chuỗi) sẽ được đưa vào đây để băm ra thành hàng loạt vi lệnh nhỏ.
- **Op-Cache**: Nếu một đoạn code cứ chạy lặp đi lặp lại (vòng lặp for/while), thay vì giải mã lại từ đầu, CPU sẽ lấy luôn kết quả đã giải mã lưu sẵn trong Op-Cache. Điều này giúp CPU chạy cực nhanh và tiết kiệm điện.

Phân xưởng 2: Middle-core (Trạm không lưu & Phân phối)

Đầu vào của khu vực này là các vi lệnh (MOPs) đổ xuống từ Decode Queue với tốc độ **12 MOPs/chu kỳ**.

1. Khối Đổi tên Thanh ghi (Register Renaming) & PRF

Phần mềm thường chỉ có vài chục thanh ghi (ví dụ RAX, RBX) để lưu biến số. Nếu nhiều lệnh cùng dùng chung một thanh ghi (chỉ vì phần mềm hết chỗ trống), CPU sẽ bị kẹt vì lệnh sau phải chờ lệnh trước xài xong thanh ghi đó.

- **576 Integer PRF & 512 Vector PRF**: Để giải quyết, CPU tạo ra hàng trăm thanh ghi vật lý (PRF - Physical Register File) bí mật bên trong. Các lệnh sẽ được "đổi tên" thanh ghi phần mềm sang thanh ghi vật lý để chúng không dẫm chân lên nhau và có thể chạy song song.
- **1280 PRRT (Physical Register Reclaim Table)**: Bảng quản lý xem thanh ghi vật lý nào đang trống, thanh ghi nào đã dùng xong để thu hồi.

2. Retire Queue / ROB (Reorder Buffer - 620 entries)

- Đây là "cửa sổ nhìn trước" của CPU. CPU sẽ tải trước tối đa **620 lệnh** vào trong ROB.
- Từ trong đám 620 lệnh hỗn độn này, lệnh nào đã có đủ dữ liệu thì CPU cho chạy trước (**Out-of-Order Execution**), không cần tuân theo thứ tự mà lập trình viên viết.
- Khi chạy xong, chúng quay lại ROB xếp hàng chờ. ROB sẽ sắp xếp lại kết quả và trả ra ngoài phần mềm đúng theo thứ tự ban đầu để đảm bảo chương trình không bị lỗi. Con số 620 là một cửa sổ cực rộng, cho phép CPU lục lọi và vắt kiệt mọi cơ hội chạy song song.

3. Khối Issue: Phân phát lệnh xuống các máy tiện, máy phay (Scheduler) bên dưới với tốc độ khủng khiếp: $24 \mu\text{s} / \text{chu kỳ}$.

Phân xưởng 3: Back-end (Nhà máy Thực thi - Các Scheduler)

Lệnh đổ xuống đây sẽ nằm chờ trong các Scheduler (Bộ lập lịch). Khi nào dữ liệu từ RAM về đủ, Scheduler sẽ đẩy lệnh vào các bộ tính toán (ALU/FPU) ngay bên dưới nó.

1. Khối Số nguyên (Integer Schedulers - Màu vàng/xanh nhạt bên trái)

Chuyên xử lý các vòng lặp, logic, cộng trừ số nguyên.

- Có **3 cụm 32-entry Schedulers** và **2 cụm 24-entry Schedulers**.
- Dưới chúng là các **ALU (Arithmetic Logic Unit)** đa năng. Một số ALU kết hợp tính toán rẽ nhánh (ALU Branch), một số chuyên làm phép Nhân/Chia (ALU MUL DIV). Sự sắp xếp này giúp phân bổ tải rất đều.

2. Khối Số thực/Vector (Floating Point Schedulers - Màu vàng/xanh lam ở giữa)

Chuyên xử lý tính toán khoa học, đồ họa, AI, và mã hóa (như lệnh AVX).

- Có tới **6 cụm 52-entry Schedulers**, cực kỳ rộng rãi.
- Các cổng này chứa **VALU** (Tính toán Vector), **VMUL** (Nhân Vector), **FP ADD** (Cộng số thực).
- Mở rộng xuống dưới là các bộ Nhân/Chia số thực chuyên sâu (FP MUL, FP DIV). Kiến trúc này cho thấy năng lực xử lý dấu phẩy động (đồ họa/AI) của nhân này là vô cùng mạnh.

3. Khối Tính toán Địa chỉ (Memory Schedulers / AGU - Màu xanh lá bên phải)

Chuyên trả lời câu hỏi: "*Dữ liệu này nằm ở tọa độ nào trong RAM?*"

- Sử dụng chung một Scheduler không lồ **128-entry**.
- Bao gồm 7 bộ AGU (Address Generation Unit) làm việc độc lập: **2 AGU Store** (tính địa chỉ để ghi), **2 AGU Unified** (hỗn hợp), và **3 AGU Load** (tính địa chỉ để đọc).

Phân xưởng 4: Memory Subsystem (Hệ thống Kho bãi & Lưu thông)

Dù CPU có tính toán nhanh đến đâu mà RAM cấp dữ liệu chậm thì cũng vứt đi. Do đó, hệ thống bộ nhớ phải cực kỳ đồ sộ.

1. Load/Store Buffers (Vùng đệm Đọc/Ghi)

- Trước khi ra RAM, lệnh phải xếp hàng ở đây. **96 Store Buffer** chờ ghi dữ liệu ra ngoài, và **160 Load Buffer** chờ đọc dữ liệu vào.
- Nếu ghi dữ liệu ra mà chưa kịp đến RAM, lệnh đọc (Load) phía sau nếu cần dữ liệu đó có thể lấy ngay từ Store Buffer để tiết kiệm thời gian (Store-to-Load forwarding).

2. L1 D-Cache (128KB - 8 way)

- Kho lưu dữ liệu ngay sát CPU. 128KB là mức dung lượng gấp 3 đến 4 lần các thiết kế truyền thống.
- Có 5 mũi tên kết nối với AGU, mỗi mũi tên có băng thông **32 Bytes/Cycle**. Tức là nó có thể đọc/ghi hàng trăm byte dữ liệu chỉ trong một nhịp đồng hồ.
- **L1 DTLB (384 entry)**: Giống ITLB ở trên, nhưng chuyển dịch địa chỉ cho "Dữ liệu" thay vì "Lệnh".

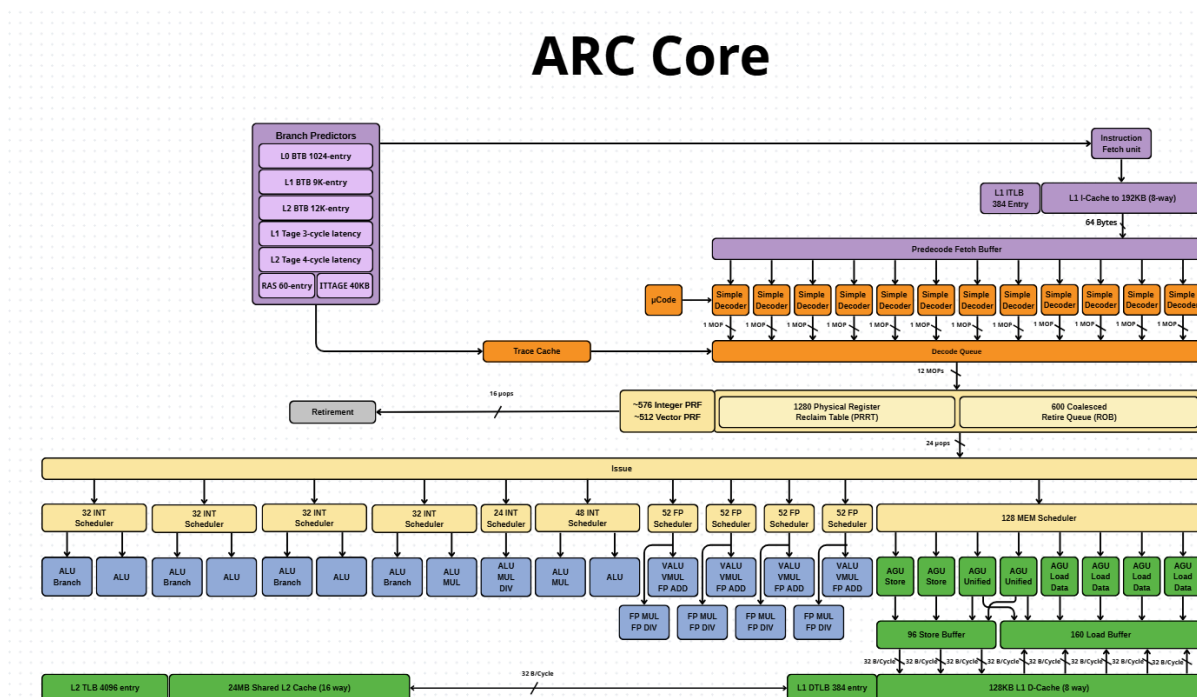
3. L2 Cache (24MB Shared - 16 way)

- L2 Cache thường nằm chung trên một cụm (cluster) cho các nhân cùng truy cập. Ở đây, 24MB là một không gian chia sẻ (Shared) cực lớn (16-way tức là dữ liệu được băm ra 16 đường để tránh kẹt xe khi tìm kiếm).
- **L2 TLB (4096 entry)**: Bộ từ điển dịch địa chỉ cực lớn, giúp CPU tra cứu được vị trí vật lý của một lượng dữ liệu khổng lồ nằm ngoài RAM mà không bị khựng lại.
- Giao tiếp giữa L1 và L2 diễn ra qua bus **32 Bytes/Cycle**.

Tóm tắt luồng đi của một đoạn code:

CPU đoán trước bạn sẽ làm gì (Branch Predictor) -> Kéo code đó về (L1 I-Cache) -> Dịch ra ngôn ngữ máy (Decoders) -> Đổi tên các biến số để không trùng lặp (PRF) -> Quăng hết vào một kho chứa lớn (ROB) -> Lệnh nào đủ dữ liệu thì ném xuống bộ tính toán (ALU/AGU) -> Lấy hoặc cất dữ liệu (L1/L2 Cache) -> Cuối cùng, trả kết quả ra màn hình cho bạn theo đúng thứ tự (Retirement).

ARC core



Chào mừng bạn đến với **ARC Core**. Nếu P Core (Performance Core) là một cỗ máy khổng lồ tối đa hóa sức mạnh thô (brute-force), thì nhìn vào vi kiến trúc của ARC Core, chúng ta thấy một thiết kế **tinh gọn hơn, thông minh hơn về mặt luồng lệnh, và tập trung mạnh vào băng thông đọc dữ liệu**.

ARC Core có nhiều điểm tương đồng với P Core, nhưng đã được cấu trúc lại (re-architected) ở những điểm nút cổ chai cực kỳ thú vị. Dưới đây là phân tích chi tiết cách "nhà máy" ARC Core vận hành và những điểm khác biệt cốt lõi.

1. Front-end: Sự xuất hiện của "Vũ khí bí mật" Trace Cache

Khu vực đầu vào của ARC Core đã được tinh chỉnh lại, giảm bớt một chút kích thước nhưng bù đắp bằng công nghệ lưu trữ lệnh nâng cao.

- **Branch Predictors (Khối Dự đoán rẽ nhánh - Tinh gọn hơn):**
 - Hệ thống dự đoán vẫn giữ nguyên thuật toán TAGE siêu việt, nhưng dung lượng các bộ đệm được thu gọn lại để tối ưu diện tích (Die area) và điện năng.
 - Ví dụ: L0 BTB giảm còn 1024-entry (so với 1280), L1 BTB còn 9K, L2 BTB còn 12K. RAS cũng giảm nhẹ xuống 60-entry. Nó bớt đi sự "phóng khoáng" của P Core nhưng vẫn là một cụm dự đoán khổng lồ so với các CPU thông thường.
- **Instruction Fetch & L1 I-Cache (Nạp lệnh - Giữ nguyên sức mạnh):**
 - Vẫn giữ nguyên kho lệnh siêu tốc **192KB L1 I-Cache** (8-way) và khả năng kéo 64 Bytes/chu kỳ.
- **Decoders (10 Simple Decoders):**
 - Độ rộng giải mã giảm từ 12 xuống còn **10 Simple Decoders**. Việc giải mã 10 lệnh mỗi nhịp đồng hồ vẫn là một con số rất khủng khiếp (Superscalar cực rộng), cung cấp thừa mứa vi lệnh (μSops) cho phần Backend.
- **Trace Cache (Điểm khác biệt lớn nhất):**
 - ARC Core **loại bỏ Op-Cache và thay bằng Trace Cache**. Đây là một sự thay đổi mang tính bản lề.
 - *Sự khác biệt:* Op-Cache chỉ lưu lại các lệnh tính đã giải mã. Còn **Trace Cache** lưu lại "Dấu vết" (Trace) – tức là luồng thực thi động của chương trình sau khi đã dự đoán rẽ nhánh. Nếu phần mềm chạy một vòng lặp phức tạp có nhiều lệnh If/Else, Trace Cache sẽ duỗi thẳng vòng lặp đó ra thành một chuỗi μSops liên tục.
 - *Lợi ích:* Khi CPU gặp lại đoạn code này, nó không cần hỏi hệ thống Dự đoán rẽ nhánh (Branch Predictor) và cũng không cần Giải mã (Decoder) nữa. Nó chỉ việc lấy luồng μSops đã duỗi thẳng từ Trace Cache đội thẳng xuống Decode Queue. Điều này tăng cường IPC (Instructions Per Cycle) cực mạnh cho các tác vụ nhiều vòng lặp.

2. Middle-core: Tối ưu không gian chờ

Trạm trung chuyển của ARC Core gần như giữ nguyên băng thông phân phối nhưng thu hẹp nhẹ cửa sổ thực thi.

- **Bộ Đôi tên Thanh ghi (PRF - Physical Register File):**
 - Giữ nguyên dung lượng khổng lồ với **~576 Integer PRF** và **~512 Vector PRF**, đảm bảo tài nguyên dồi dào để các lệnh không bị kẹt vì thiếu thanh ghi ảo.
- **Retire Queue / ROB (600 entries):**
 - Cửa sổ nhìn trước (Out-of-Order window) được thu hẹp nhẹ từ 620 xuống **600 lệnh**. Dù vậy, sức chứa 600 lệnh chờ xếp hàng vẫn xếp ARC Core vào hạng mục vi kiến trúc cực kỳ sâu (deep-pipeline), cho phép xáo trộn và chạy song song lượng lớn dữ liệu.

- **Issue:** Vẫn giữ tốc độ đẩy lệnh kinh hoàng: **24 μ Sops** / chu kỳ xuống các bộ lập lịch.

3. Back-end: Cơ bắp sẵn chắc hơn

Khu vực thực thi (Execution Engine) được cắt giảm số lượng cổng (ports) để tiết kiệm điện năng và diện tích silicon, nhưng vẫn phân bổ logic rất thông minh.

- **Khối Số nguyên (Integer Schedulers - 6 Cụm):**
 - Tăng từ 5 lên **6 Scheduler**.
 - Cách bố trí có sự phân hóa rõ rệt về độ sâu: Có 4 cụm 32-entry, 1 cụm 24-entry (chuyên trị Phép Nhân/Chia ALU MUL DIV), và đặc biệt có **1 cụm 48-entry** lớn hơn hẳn để quản lý ALU kết hợp MUL. Sự bất đối xứng này cho thấy ARC Core được thiết kế để ưu tiên luân chuyển các lệnh số nguyên phức tạp vào các hàng đợi sâu hơn.
- **Khối Số thực/Vector (Floating Point Schedulers - 4 Cụm):**
 - Giảm từ 6 xuống **4 Scheduler** (mỗi cái 52-entry).
 - Các cổng này vẫn trang bị đầy đủ VALU (Vector ALU), VMUL và FP ADD, bên dưới là các bộ FP MUL/DIV chuyên dụng. Với 4 cổng, năng lực xử lý AVX/AI/Đồ họa của ARC Core yếu hơn P Core một chút, nhưng đổi lại nhân CPU sẽ mát hơn và chiếm ít diện tích lõi hơn rất nhiều.

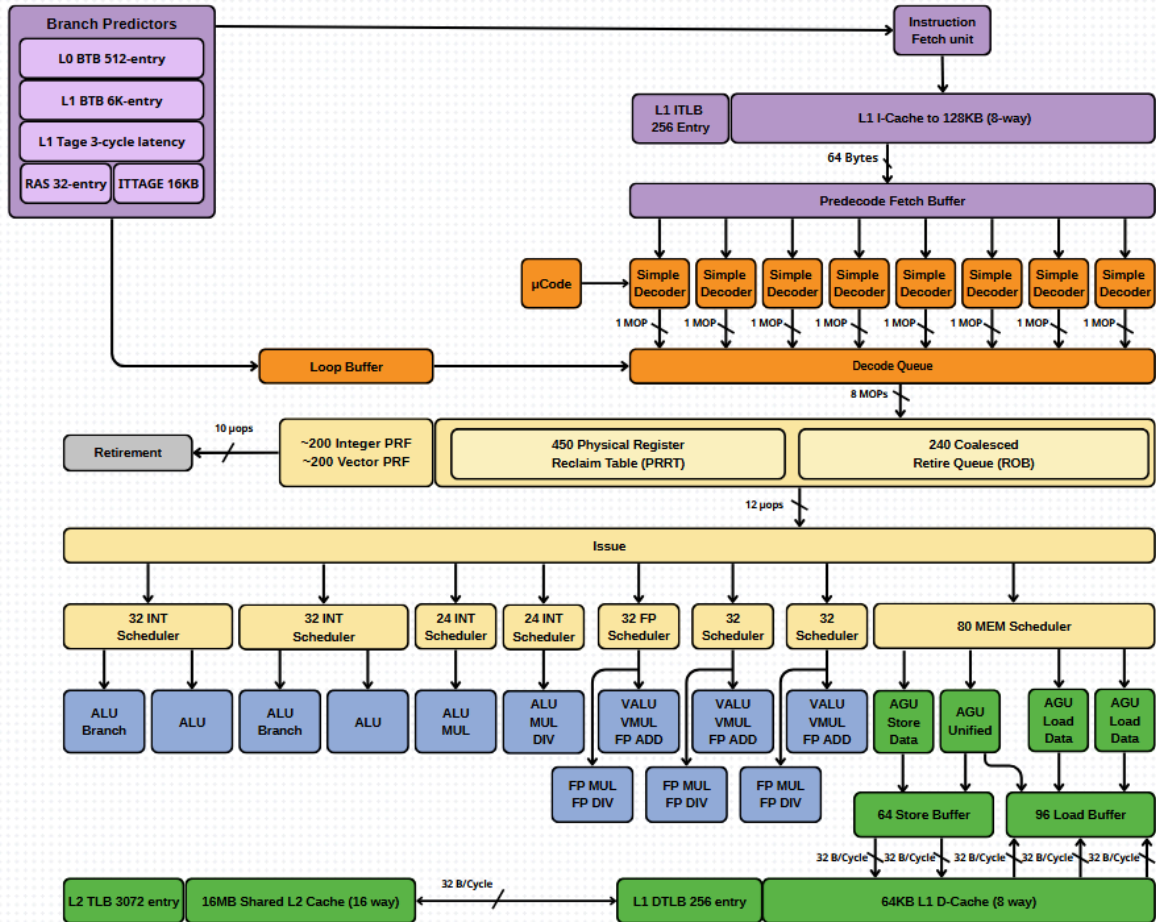
4. Memory Subsystem: Quái vật "Hút" dữ liệu

Đây là nơi ARC Core tỏa sáng và thể hiện triết lý thiết kế riêng biệt của mình. Nó hy sinh một chút cơ bắp tính toán để nâng cấp đường ống hút dữ liệu.

- **128 MEM Scheduler & Các cổng AGU:**
 - Bộ lập lịch bộ nhớ giữ nguyên 128-entry.
 - **Nâng cấp AGU:** Mặc dù số lượng cổng tính toán giảm, nhưng cổng bộ nhớ của ARC Core lại **tăng từ 7 lên 8 AGU**.
 - Cụ thể: 2 cổng Store, 2 cổng Unified, và **4 cổng Load Data** (P Core chỉ có 3).
 - *Ý nghĩa:* ARC Core có thể tính toán và đọc 4 khối dữ liệu từ RAM/Cache cùng một lúc trong một chu kỳ nhịp đồng hồ. Điều này biến nó thành một con quái vật xử lý các tác vụ Data-heavy (phân tích dữ liệu, database), nơi CPU dành phần lớn thời gian để "đọc" dữ liệu vào xử lý thay vì "ghi" kết quả ra.
- **Load / Store Buffers & L1 D-Cache:**
 - Vẫn trang bị vùng đệm khổng lồ (96 Store / 160 Load).
 - Bộ nhớ đệm **128KB L1 D-Cache** tiếp tục phát huy sức mạnh với băng thông rộng (32 B/Cycle) để phục vụ kịp thời 4 cổng Load Data đang đói dữ liệu ở trên.
- **L2 Cache (24MB Shared):** Khối L2 chung khổng lồ tiếp tục đảm bảo hệ thống hiếm khi phải truy xuất ra ngoài RAM chậm chạp.

C core

C Core



1. Front-end: Cắt giảm dự đoán, thêm "Loop Buffer" tiết kiệm điện

Khu vực này bị thu nhỏ đáng kể để nhường chỗ cho khả năng tiết kiệm năng lượng.

- **Branch Predictors (Khởi Dự đoán rẽ nhánh - Bị cắt giảm mạnh):**
 - **L0 BTB:** Chỉ còn **512-entry** (giảm hơn một nửa so với P Core).
 - **L1 BTB:** Chỉ còn **6K-entry**.
 - **Đặc biệt:** Hoàn toàn **không có L2 BTB và L2 TAGE**. TAGE bây giờ chỉ còn L1 với độ trễ 3 chu kỳ. RAS bị ép xuống còn **32-entry** và ITTAGE còn **16KB**.
 - *Đánh giá:* CPU không còn nhìn xa trông rộng được như trước. Nó chấp nhận đoán sai nhiều hơn một chút để đổi lấy việc thu nhỏ kích thước nhân.
- **Instruction Fetch:** L1 I-Cache cũng bị cắt giảm từ 192KB xuống còn **128KB** (8-way). L1 ITLB giảm còn **256 entry**. Băng thông nạp lệnh vẫn duy trì ở 64 Bytes.
- **Decoders (Khởi giải mã):**
 - Chỉ còn **8 Simple Decoders** (thay vì 12 hay 10). Băng thông đẩy vi lệnh xuống giảm còn **8 MOPs/chu kỳ**.
- **Vũ khí tối ưu điện: Loop Buffer:**

- C Core không dùng Op-Cache đắt đỏ hay Trace Cache phức tạp. Nó dùng một **Loop Buffer**.
- *Cách hoạt động*: Khi CPU gặp một vòng lặp nhỏ (ví dụ lặp lại 100 lần một phép cộng), Loop Buffer sẽ khóa luồng lệnh đó lại và phát trực tiếp μSops xuống dưới. Trong lúc đó, toàn bộ phần Fetch, Predictor và Decoder ở trên có thể... "tắt điện đi ngủ" để tiết kiệm pin. Đây là kỹ thuật kinh điển của các nhân E-Core.

2. Middle-core: Thu hẹp "Cửa sổ nhìn trước"

Trạm trung chuyển dữ liệu được thu nhỏ lại để vừa vặn với lượng lệnh ít ỏi hơn từ Front-end đổ xuống.

- **PRF (Thanh ghi vật lý)**: Dung lượng giảm cực sâu, chỉ còn **~200 Integer PRF** và **~200 Vector PRF** (so với hơn 500 của P/ARC). PRRT cũng chỉ còn 450.
- **Retire Queue / ROB (Trái tim của Out-of-Order)**:
 - Chỉ còn **240 entries** (so với 620 của P Core).
 - Cửa sổ 240 lệnh này khiến C Core không thể nhặt ra quá nhiều lệnh để chạy song song, nhưng bù lại logic kiểm soát sẽ đơn giản hơn, tốn ít điện hơn rất nhiều.
- **Issue Width**: Băng thông phân phối lệnh xuống Backend bị chém một nửa, chỉ còn **12 μSops /chu kỳ** (thay vì 24). Đầu ra (Retirement) dọn dẹp với tốc độ **10 μSops** .

3. Back-end: Cơ bắp thu gọn gọn gàng

Động cơ thực thi được tinh giản tối đa, loại bỏ sự dư thừa, chỉ giữ lại những gì cần thiết nhất để chạy các tác vụ nền (background tasks) hoặc đa luồng cơ bản.

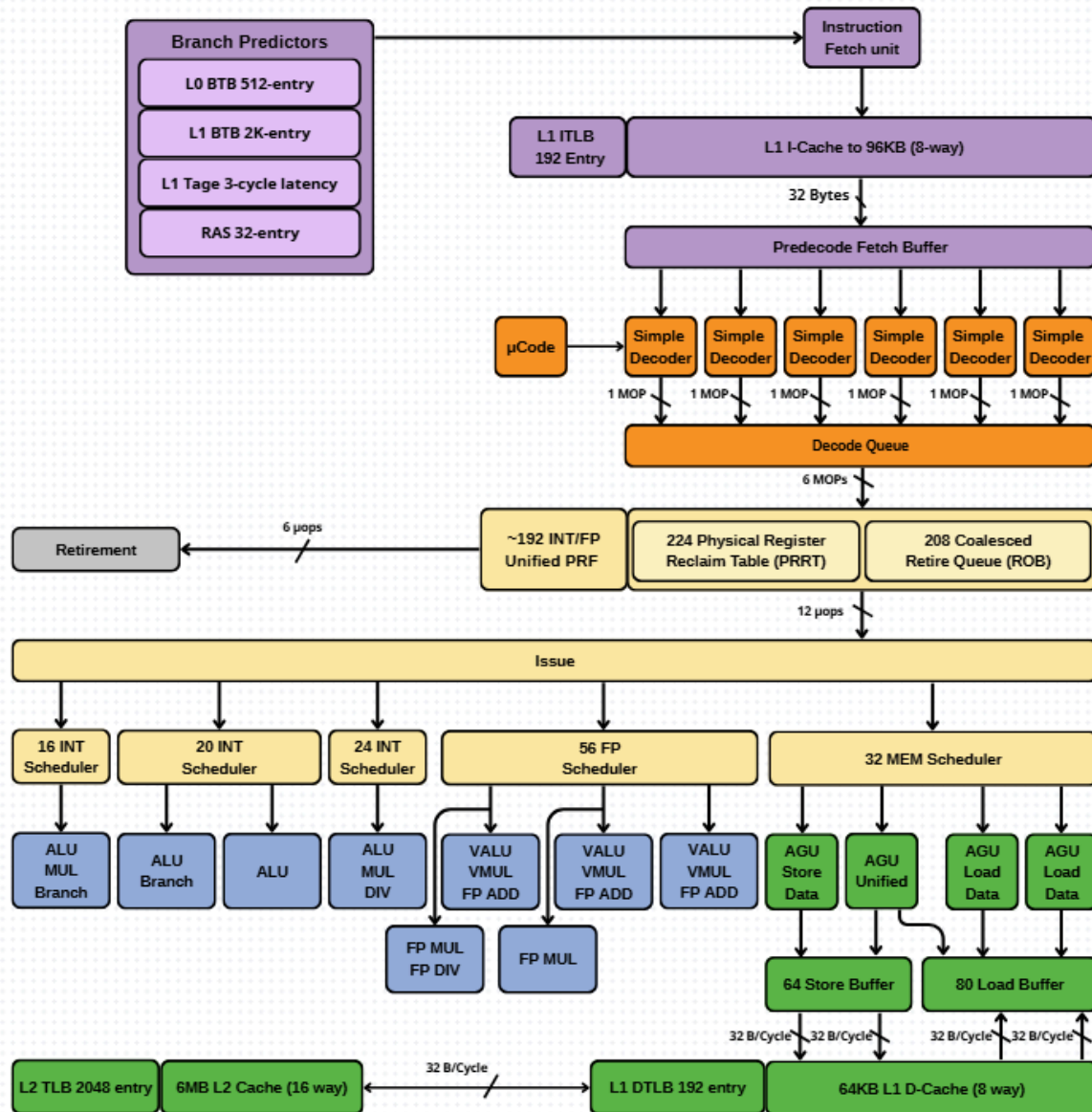
- **Khối Số nguyên (Integer Schedulers - 4 Cụm)**:
 - Có **2 cụm 32-entry** và **2 cụm 24-entry** (tổng cộng 4 Scheduler).
 - Cung cấp năng lượng cho 6 ALU bên dưới. Trong đó có 2 ALU Branch, 2 ALU thuần, 1 ALU MUL và 1 ALU MUL DIV.
- **Khối Số thực/Vector (Floating Point Schedulers - 3 Cụm)**:
 - Giảm xuống chỉ còn **3 Scheduler** (đồng cỡ 32-entry).
 - Vẫn hỗ trợ các lệnh Vector cơ bản (VALU, VMUL, FP ADD, FP MUL/DIV) nhưng năng lực tính toán song song bị giới hạn bởi số lượng cổng.
- **Khối Bộ nhớ (Memory Schedulers / AGU)**:
 - Dùng chung 1 Scheduler chứa **80-entry**.
 - Chỉ còn **4 AGU** (so với 8 của ARC Core): Bao gồm 1 AGU Store Data, 1 AGU Unified, và 2 AGU Load Data. Tốc độ đọc/ghi bị bóp hẹp rõ rệt.

4. Memory Subsystem: Hệ thống đệm vừa đủ dùng

- **Buffers chờ**: Bị thu nhỏ lại còn **64 Store Buffer** và **96 Load Buffer**.
- **L1 D-Cache**: Bị cắt làm đôi, chỉ còn **64KB (8-way)**. Số lượng liên kết băng thông (32 B/Cycle) cũng giảm tương ứng với số cổng AGU (còn 4 liên kết). L1 DTLB là 256 entry.
- **L2 Cache (16MB Shared)**: Khối L2 dùng chung giảm từ 24MB xuống **16MB (16-way)**, đi kèm với L2 TLB 3072 entry. Băng thông giao tiếp L1-L2 là 32 B/Cycle.

E core

E Core



1. Front-end: Cắt giảm tối đa "tầm nhìn" và "băng thông"

Khu vực nạp lệnh của E Core bị bóp nghẹt lại chỉ còn đủ để duy trì một luồng lệnh cơ bản, không có các cơ chế dự đoán xa xỉ.

- **Branch Predictors (Khởi Dự đoán rẽ nhánh):**
 - **L0 BTB:** 512-entry (giữ nguyên như C Core).
 - **L1 BTB:** Bị chém mạnh tay chỉ còn **2K-entry** (so với 6K của C Core và 12K của P Core).
 - **L1 TAGE:** Vẫn giữ độ trễ 3 chu kỳ nhưng không có L2 TAGE.
 - **RAS:** 32-entry.

- *Đặc biệt: Mất hoàn toàn ITTAGE.* E Core gần như từ bỏ việc dự đoán các điểm rẽ nhánh gián tiếp phức tạp để tiết kiệm các bóng bán dẫn (transistor).
- **Instruction Fetch & L1 I-Cache:**
 - Dung lượng bộ nhớ đệm lệnh giảm còn **96KB (8-way)**.
 - Bảng thông kéo lệnh từ Cache bị bóp làm đôi, chỉ còn **32 Bytes** (các core trước là 64 Bytes). Bộ dịch địa chỉ L1 ITLB cũng giảm còn 192 Entry.
- **Decoders (Khởi giải mã):**
 - Chỉ còn **6 Simple Decoders**. Bảng thông đầu ra chỉ là **6 MOPs/chu kỳ** (bằng một nửa so với P Core). Điều này cho thấy kiến trúc này cực kỳ hẹp (narrow-width).

2. Middle-core: Kiến trúc "Hộp nhất" để tiết kiệm diện tích

Trái tim của hệ thống phân phối lệnh có một sự thay đổi mang tính bước ngoặt về mặt kiến trúc để tiết kiệm không gian.

- **Unified PRF (Điểm nhấn lớn nhất):**
 - Thay vì tách riêng thanh ghi cho số nguyên (Integer) và số thực (Vector/FP) như các core trước, E Core dùng chung một bộ **~192 INT/FP Unified PRF** (Thanh ghi vật lý hộp nhất).
 - Việc dùng chung này giúp giảm rất nhiều hệ thống dây dẫn và logic điều khiển, đôi lại CPU có thể bị nghẽn nếu một phần mềm dùng quá nhiều lệnh đồ họa và toán học cùng lúc. PRRT đi kèm cũng nhỏ bé với 224 mục.
- **Retire Queue / ROB (Trái tim Out-of-Order):**
 - Cửa sổ thực thi thu hẹp lại mức "đầy", chỉ còn **208 entries**. CPU không thể nhìn quá xa để tìm lệnh chạy song song được nữa.
- **Bảng thông nội bộ:** Tốc độ đẩy lệnh xuống bộ lập lịch là 12 μ Sops, nhưng tốc độ nghiệm thu và dọn dẹp (Retirement) chỉ đạt 6 μ Sops/chu kỳ.

3. Back-end: Cơ bắp bị "teo nhỏ" nhưng vẫn đủ xài

Sức mạnh tính toán bị giới hạn nghiêm ngặt, cách bố trí hàng đợi (Queue) cũng rất bất đối xứng.

- **Khối Số nguyên (Integer Schedulers - 3 Cụm):**
 - E Core chia bộ lập lịch số nguyên thành 3 khối nhỏ lẻ tẻ với kích thước không đều: **16-entry, 20-entry, và 24-entry**.
 - Dù nhỏ, nó vẫn cung cấp đủ các cổng ALU đa năng: 2 cổng ALU Branch, 1 cổng ALU thuần, và 2 cổng hỗ trợ Nhân/Chia (MUL/DIV). Điều này đảm bảo nó vẫn xử lý tốt các tác vụ logic và hệ điều hành.
- **Khối Số thực/Vector (Floating Point Scheduler - Hộp nhất):**
 - Chỉ có **1 Scheduler duy nhất (56-entry)** gánh vác toàn bộ mảng tính toán số thập phân và Vector.
 - Nó phân phối xuống 3 cổng VALU/VMUL/FP ADD. Khả năng chạy các lệnh nặng như AVX của nhân này sẽ rất chậm và mang tính "cho có" hơn là để cây cuốc.

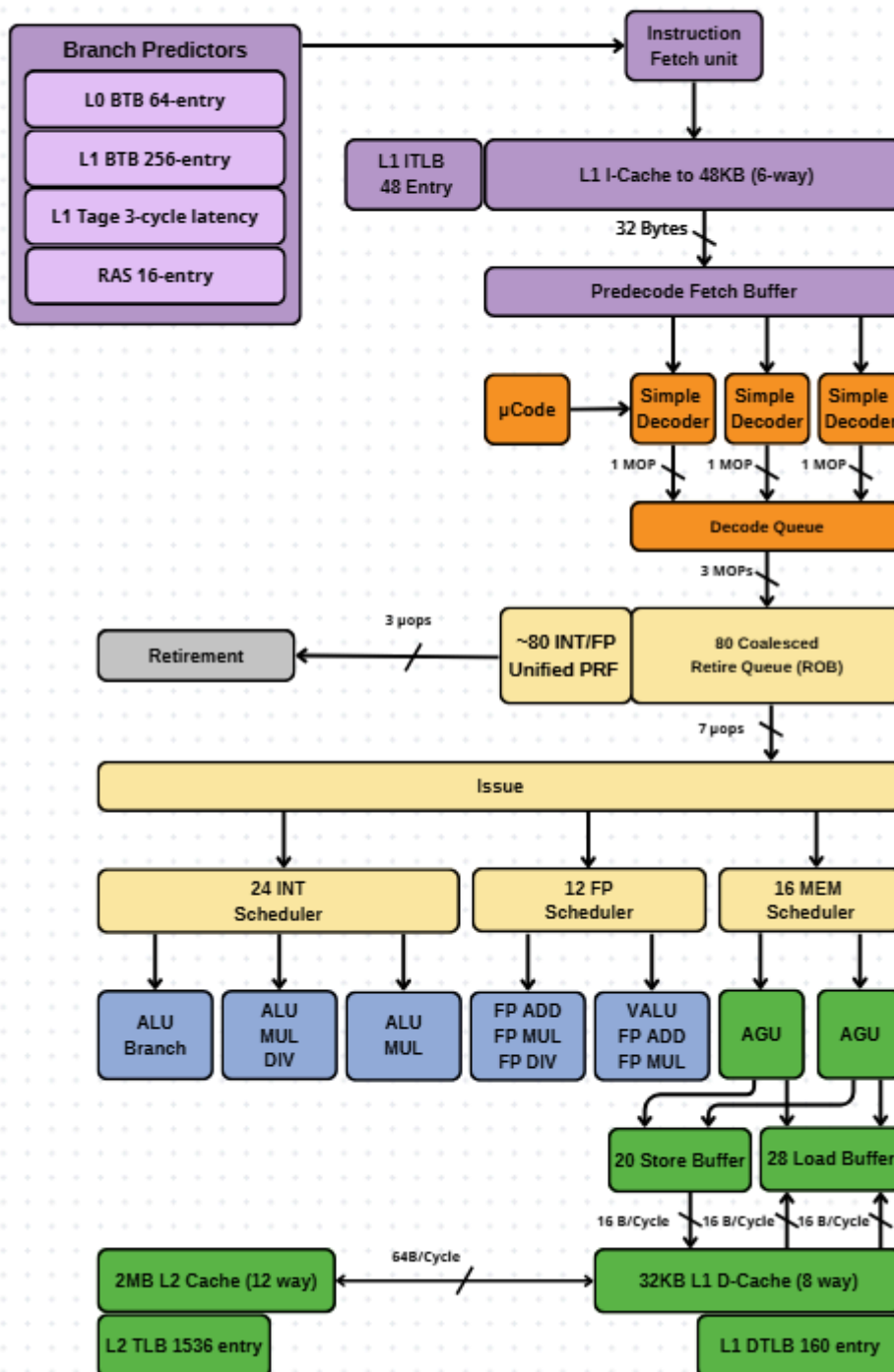
4. Memory Subsystem: Giữ vững tốc độ truyền tải cơ bản

Bảng thông bộ nhớ dù bị cắt giảm nhưng vẫn được thiết kế để không trở thành điểm nghẽn của một CPU vốn đã chậm.

- **MEM Scheduler & AGU:**
 - Bộ lập lịch bộ nhớ chỉ còn **32-entry**.
 - Vẫn giữ **4 AGU** (1 Store, 1 Unified, 2 Load Data) giống hệ C Core. Tuy nhiên, hàng đợi Load/Store Buffer đã bị ép nhỏ lại còn **64 Store / 80 Load**.
- **L1 D-Cache:** Duy trì mức **64KB (8-way)**, bảng thông giao tiếp nội bộ vẫn giữ 32 B/Cycle. L1 DTLB là 192 entry.
- **L2 Cache (Bộ nhớ đệm cấp 2):**
 - Khối L2 Cache giảm cực sâu, chỉ còn **6MB (16-way)** với L2 TLB 2048 entry.

LPE core

LPE Core



Chào bạn, chúng ta đã đi đến thiết kế cuối cùng và cũng là thiết kế "nhỏ bé" nhất trong dải vi kiến trúc này: LPE Core (Low Power Efficiency Core - Nhân Tiết kiệm điện năng thấp).

Nếu E Core là một thiết kế "ăn kiêng", thì LPE Core thực sự là một thiết kế "**duy trì sự sống**" (**Always-on**). Khái niệm này cực kỳ phổ biến trong các kiến trúc SoC hiện đại (giống như khái niệm *Low Power Island*), nơi nhân này được đặt ở một phân vùng nguồn điện riêng biệt để gánh vác các tác vụ nền khi toàn bộ hệ thống đang ngủ say.

Mọi thông số của LPE Core đều được ép xuống mức tối thiểu, đưa nó về ranh giới mong manh giữa kiến trúc Out-of-Order (thực thi không theo thứ tự) và In-Order (thực thi tuần tự cổ điển).

Dưới đây là màn mổ xẻ chi tiết "hạt tiêu" LPE Core này:

1. Front-end: Tối giản hóa đến mức tối đa

Hệ thống nạp và giải mã lệnh của LPE Core chỉ giữ lại những gì cơ bản nhất để CPU có thể hoạt động, loại bỏ hoàn toàn khả năng "nhìn xa trông rộng".

- **Branch Predictors (Khởi Dự đoán rẽ nhánh - Tí hon):**
 - **L0 BTB:** Chỉ có vốn vẹn **64-entry**.
 - **L1 BTB:** Rút gọn còn **256-entry** (bằng 1/8 so với E Core).
 - Vẫn giữ thuật toán **L1 TAGE** (3 chu kỳ) để bù đắp lại dung lượng bộ đệm quá nhỏ, và **RAS 16-entry**. Bộ phận dự đoán bây giờ chỉ mang tính chất "đoán được chừng nào hay chừng đó" thay vì bao quát toàn bộ luồng chương trình.
- **Instruction Fetch & L1 I-Cache:**
 - Bộ nhớ đệm lệnh có mức dung lượng khá là: **48KB (6-way)**. Đây là một con số được căn chỉnh cực kỳ chi li để cân bằng giữa diện tích silicon và tỷ lệ hit-rate của Cache.
 - Băng thông kéo lệnh là **32 Bytes**, L1 ITLB siêu nhỏ với **48 Entry**.
- **Decoders (Chỉ còn 3 bộ giải mã):**
 - Từ 12 bộ của P Core, xuống 6 của E Core, nay LPE Core chỉ còn **3 Simple Decoders**.
 - Băng thông đầu ra chỉ đạt **3 MOPs/chu kỳ**. Điều này đồng nghĩa LPE Core là một thiết kế cực kỳ "hẹp" (narrow-width), tiêu tốn cực ít điện nhưng cũng nhai lệnh rất chậm.

2. Middle-core: Vừa đủ để gỡ rối luồng lệnh

Trạm trung chuyển dữ liệu bị thu hẹp đến mức giới hạn, vừa đủ để luồng lệnh không bị tắc nghẽn hoàn toàn.

- **ROB (Retire Queue) siêu nông:**
 - Cửa sổ thực thi Out-of-Order chỉ có **80 entries**. Với 80 lệnh nằm chờ, CPU gần như không có nhiều cơ hội để xáo trộn và tìm kiếm các lệnh chạy song song. Nó hoạt động gần giống một CPU In-Order truyền thống nhưng mượt mà hơn một chút.
- **Unified PRF (Thanh ghi hợp nhất):**
 - Chỉ có **~80 INT/FP Unified PRF**. Khối lượng thanh ghi ảo này đủ để phục vụ cho cửa sổ 80 lệnh của ROB ở trên.
- **Băng thông nội bộ:** Tốc độ nghiệm thu (Retirement) giảm xuống mức cơ sở là **3 μ ops/chu kỳ**, khớp hoàn toàn với tốc độ giải mã đầu vào.

3. Back-end: Cơ bắp thu nhỏ phục vụ tác vụ nền

Động cơ thực thi không còn chia thành nhiều cụm nhỏ như E Core nữa mà gộp chung lại để dùng chung logic lập lịch.

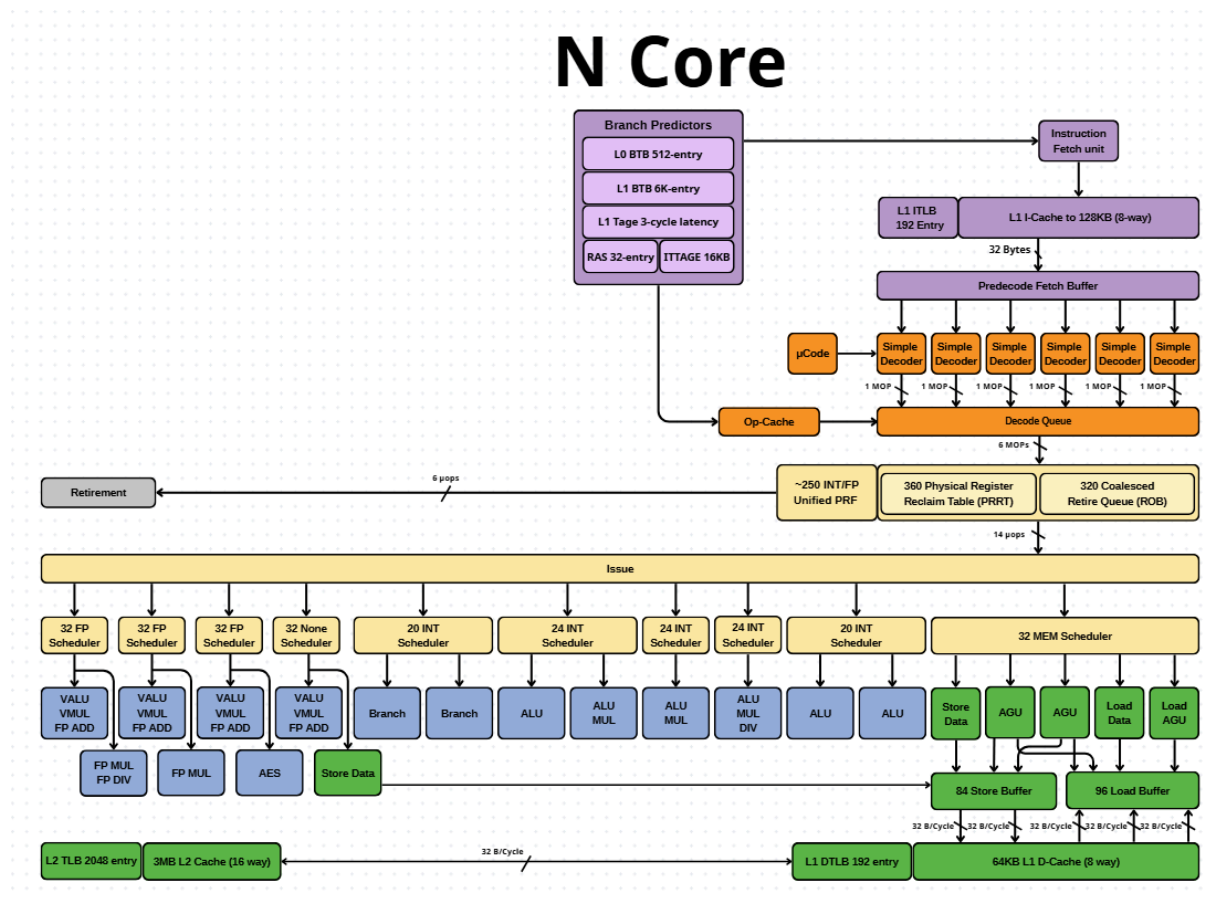
- **Khối Số nguyên (24-entry INT Scheduler):**
 - Sử dụng duy nhất 1 bộ lập lịch 24-entry để nuôi 3 cổng tính toán: 1 cổng ALU Branch, 1 cổng ALU MUL DIV, và 1 cổng ALU MUL.
 - Sức mạnh này thừa sức xử lý các vòng lặp kiểm tra cảm biến, duy trì kết nối mạng nền hoặc phát nhạc audio.
- **Khối Số thực/Vector (12-entry FP Scheduler):**
 - Cực kỳ nhỏ gọn với hàng đợi chỉ 12 lệnh.
 - Chỉ có 2 cổng thực thi: 1 cổng chuyên tính toán (FP ADD/MUL/DIV) và 1 cổng Vector hỗn hợp. Khối này tồn tại chủ yếu để đảm bảo tính tương thích của tập lệnh (ISA) chứ không dùng để xử lý nặng.

4. Memory Subsystem: Thiết kế cổ chai có chủ đích

Hệ thống lấy dữ liệu được thiết kế lại, hạn chế băng thông xuống các cổng tính toán nhưng lại mở rộng băng thông ra L2 Cache.

- **MEM Scheduler (16-entry) & 2 AGU:**
 - Bộ lập lịch bộ nhớ chỉ có 16 chỗ trống, điều khiển **2 cổng AGU**.
 - Store Buffer và Load Buffer bị ép xuống mức rất thấp: **20 Store / 28 Load**.
- **L1 D-Cache (32KB - 8-way):**
 - Dung lượng thu nhỏ còn 32KB.
 - Đáng chú ý: Băng thông nội bộ từ AGU xuống L1 Data chỉ là **16 B/Cycle** (các core khác đều là 32 B/Cycle). Nó giới hạn lượng dữ liệu có thể nhồi vào CPU mỗi chu kỳ để tránh vượt quá công suất tiêu thụ (TDP) cho phép.
- **L2 Cache (2MB - 12 way):**
 - Mặc dù nhân rất yếu, nhưng khối L2 lại khá ổn (2MB) kèm L2 TLB 1536 entry.
 - Đặc biệt, kết nối giữa L1 và L2 lại có băng thông rất lớn: **64 B/Cycle**. Thiết kế này ngụ ý rằng LPE Core thường xuyên phải đẩy/kéo các khối dữ liệu (cache lines) lớn vào L2 để chia sẻ với các nhân lớn hơn (như P Core/C Core) khi hệ thống thức giấc, đóng vai trò như một trạm trung chuyển dữ liệu nền.

N core



1. Front-end: Nút thắt cổ chai và "Vũ khí bí mật"

Khu vực này của N Core là một sự pha trộn kỳ lạ giữa sự cắt giảm của E Core và công nghệ đắt tiền của P Core.

- **Branch Predictors (Khối Dự đoán rẽ nhánh - Tương đương C Core):**
 - Hệ thống dự đoán khá tiêu chuẩn: **L0 BTB 512-entry**, **L1 BTB 6K-entry**, **L1 TAGE 3-cycle**, **RAS 32-entry**, và **ITTAGE 16KB**. Tầm nhìn của nó xa hơn E Core nhưng không bằng P/ARC.
- **Instruction Fetch & L1 I-Cache:**
 - Bộ nhớ đệm lệnh lớn **128KB (8-way)**, đi kèm **L1 ITLB 192 Entry**. Tuy nhiên, băng thông kéo lệnh bị bóp lại chỉ còn **32 Bytes** (giống E Core).
- **Decoders (Khối giải mã - Tương đương E Core):**
 - Chỉ có **6 Simple Decoders**, cho ra băng thông giải mã hẹp **6 MOPs/chu kỳ**.
- **Vũ khí bí mật: Op-Cache:**
 - Đây là điểm bất ngờ nhất! C Core và E Core hoàn toàn loại bỏ Op-Cache để tiết kiệm diện tích, nhưng N Core lại **mang Op-Cache trở lại**.
 - *Ý nghĩa chiến lược:* Dù khối Decoder chỉ giải mã được 6 lệnh/chu kỳ (rất chậm), nhưng nếu phần mềm chạy vào một vòng lặp (loop) đã được giải mã sẵn trong Op-Cache, luồng vi lệnh sẽ được xả thẳng xuống dưới với tốc độ cực cao, bỏ qua

hoàn toàn sự chậm chạp của Decoder. Thiết kế này tối ưu cực tốt cho các đoạn code lặp đi lặp lại nhiều lần.

2. Middle-core: Cửa sổ thực thi sâu bất ngờ

Dù Front-end hẹp, nhưng khu vực chứa lệnh chờ của N Core lại được làm rất rộng, cho thấy kiến trúc này được sinh ra để cày cuốc dữ liệu (throughput).

- **Retire Queue / ROB (320 entries):**
 - Cửa sổ Out-of-Order lên tới **320 lệnh**. Con số này lớn hơn hẳn C Core (240) và E Core (208). Nó cho phép N Core xáo trộn và tìm kiếm cơ hội chạy song song trên một tập lệnh lớn hơn rất nhiều.
- **Unified PRF (Thanh ghi hợp nhất):**
 - Sử dụng thanh ghi dùng chung **~250 INT/FP Unified PRF** kèm PRRT 360 mục. Con số này đủ lớn để "nuôi" cửa sổ ROB 320 lệnh ở trên.
- **Băng thông nội bộ:** Tốc độ đẩy lệnh (Issue) xuống Schedulers khá dư dả ở mức **14 μ ops**, trong khi tốc độ dọn dẹp (Retirement) giữ ở mức cơ sở **6 μ ops**.

3. Back-end: Cơ bắp chuyên biệt (Cryptography & Vector)

N Core lộ rõ bản chất là một "nhân chuyên trị" (Accelerator Core) khi nhìn vào cách bố trí các cổng thực thi. Nó đầu tư rất mạnh vào toán học và bảo mật.

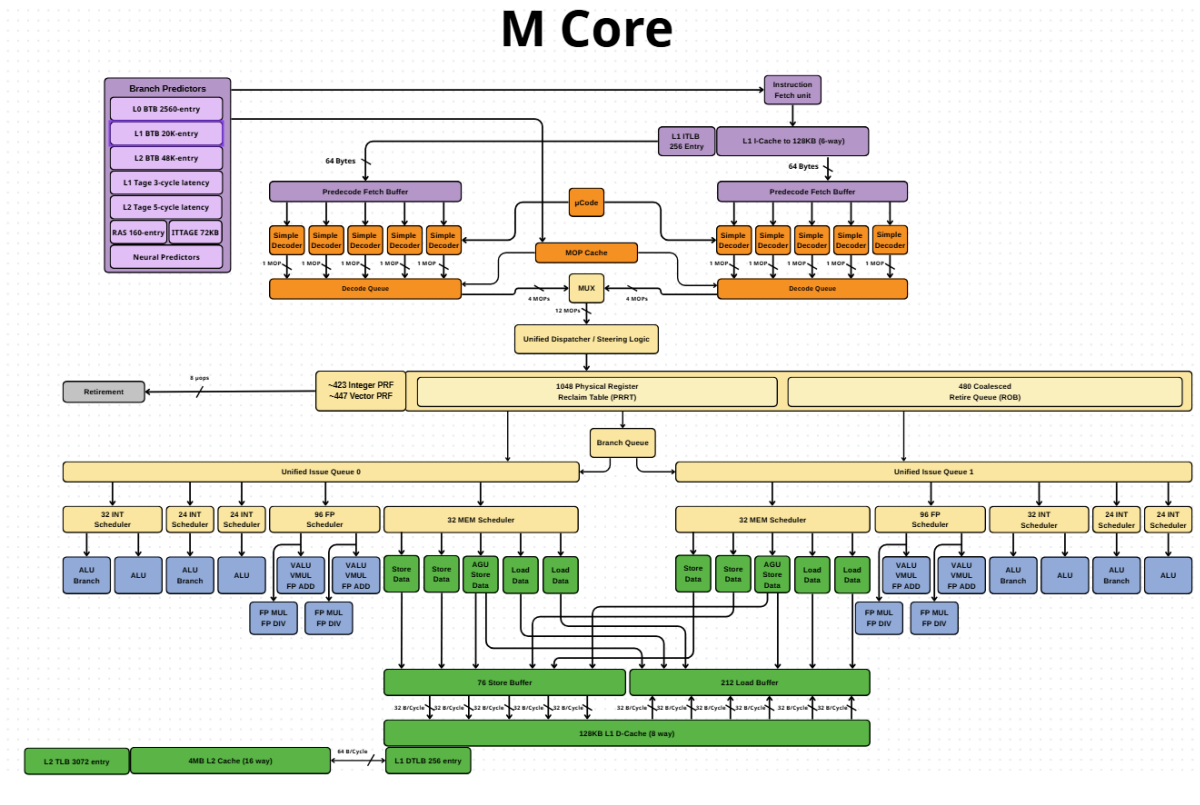
- **Khối Số nguyên (Integer Schedulers - 5 Cụm):**
 - Có tới **5 bộ lập lịch** (hai bộ 20-entry và ba bộ 24-entry), nhiều hơn cả C Core (chỉ có 4).
 - Chúng cung cấp logic cho một dàn ALU đa dạng: từ ALU xử lý rẽ nhánh, ALU thuần, đến các ALU hỗ trợ Nhân/Chia (MUL/DIV). Năng lực xử lý số nguyên của nhân này rất đáng gờm.
- **Khối Số thực/Vector (FP Schedulers - 4 Cụm & Cổng AES):**
 - Có **4 bộ lập lịch 32-entry** (trong khi E Core chỉ có 1).
 - Ngoài các bộ tính toán Vector tiêu chuẩn (VALU, VMUL, FP ADD, FP MUL/DIV), điểm nhấn cực mạnh là sự xuất hiện của **cổng AES chuyên dụng** (Advanced Encryption Standard).
 - *Ý nghĩa:* N Core có khả năng giải mã/mã hóa dữ liệu bảo mật (crypto) và tính toán vector cực kỳ mạnh mẽ mà không cần huy động đến P Core tốn điện. Thêm vào đó, khối này còn có một đường **Store Data** riêng rẽ kết nối thẳng ra bộ nhớ.

4. Memory Subsystem: Hệ thống đệm tùy biến

- **MEM Scheduler & AGU:**
 - Bộ lập lịch 32-entry điều khiển 5 khối AGU (Store Data, 2x AGU hỗn hợp, Load Data, Load AGU).
 - Load/Store Buffer ở mức vừa phải: **84 Store / 96 Load**.
- **L1 D-Cache & L2 Cache:**
 - L1 Data duy trì mức **64KB (8-way)** với băng thông nội bộ 32 B/Cycle.

- Khối **L2 Cache có dung lượng 3MB (16-way)** kèm L2 TLB 2048 entry. Con số 3MB này là một mức cắt giảm rất "vừa vặn" (không quá nhỏ như LPE, không quá lớn như E/C), đủ để chứa các khối dữ liệu Vector/Crypto đang xử lý dở dang.

M core



1. Front-end: "Bộ não phân đôi" và Trí tuệ Nhân tạo

Khu vực đầu vào của M Core là lớn nhất, phức tạp nhất và tốn nhiều silicon nhất trong tất cả các core bạn từng thiết kế.

- **Branch Predictors (Khối Dự đoán - Tích hợp AI):**
 - Đây là một cụm dự đoán khổng lồ vượt xa cả P Core: **L0 BTB 2560-entry** (gấp đôi P Core), **L1 BTB 20K**, và **L2 BTB 48K-entry** siêu to. Đi kèm TAGE 2 cấp độ và ITTAGE 72KB.
 - **Điểm nhấn: Neural Predictors.** M Core chính thức đưa Mạng nơ-ron (AI) vào việc dự đoán rẽ nhánh. AI sẽ học thói quen của phần mềm theo thời gian thực để đoán trước đường đi với tỷ lệ chính xác gần như tuyệt đối, hạn chế tối đa việc đoán sai làm sập toàn bộ hệ thống "hai đầu" bên dưới.
- **Instruction Fetch & L1 I-Cache:**
 - L1 I-Cache giữ mức **128KB (6-way)**, nhưng điểm đặc biệt là nó xuất ra **2 luồng 64 Bytes song song**.
- **Khối Decoders & MOP Cache (Hệ thống giải mã kép):**

- M Core không dùng 1 hàng Decoder dài, mà chia làm **2 cụm độc lập**, mỗi cụm có **5 Simple Decoders**. Điều này cực kỳ lý tưởng để giải mã 2 luồng phần mềm cùng lúc (SMT).
- Ổ giữa là **MOP Cache** (lưu các lệnh đã giải mã).
- **Bộ MUX**: Hệ thống trộn và lựa chọn luồng. Nó có thể rút 4 MOPs từ cụm trái, 4 MOPs từ cụm phải, kết hợp với MOP Cache để đội xuống một lượng khổng lồ **12 MOPs / chu kỳ**.

2. Middle-core: Logic điều hướng và Cửa sổ thực thi

Trạm trung chuyển của M Core được thiết kế lại hoàn toàn để phục vụ cho kiến trúc cụm kép ở Back-end.

- **Unified Dispatcher / Steering Logic (Khối Điều hướng hợp nhất):**
 - Đây là "người chia bài". Nhận 12 MOPs từ trên xuống, nó sẽ phân tích xem lệnh nào nên đẩy sang Cụm Trái (Cluster 0), lệnh nào đẩy sang Cụm Phải (Cluster 1) để cân bằng tải và tránh xung đột dữ liệu.
- **PRF & ROB:**
 - Dung lượng thanh ghi khá khiêm tốn so với P Core: **~423 Integer PRF** và **~447 Vector PRF**.
 - Cửa sổ **ROB là 480 entries**. Tốc độ Retirement dọn dẹp là **8 μops**.
 - *Tại sao ROB lại nhỏ hơn P Core?* Vì với kiến trúc chia cụm này, CPU dựa vào việc "tẩy" lệnh cực nhanh qua hai động cơ song song thay vì giữ lệnh lại quá lâu để tìm kiếm.
- **Branch Queue & Unified Issue Queue 0 / 1:**
 - Lệnh sau khi đổi tên sẽ được rẽ làm đôi xuống **Issue Queue 0** (Trái) và **Issue Queue 1** (Phải).
 - Ổ giữa có một **Branch Queue** chung để quản lý các lệnh rẽ nhánh của cả 2 bên.

3. Back-end: "Cặp song sinh" Hủy diệt

Thay vì làm một khối Scheduler khổng lồ (rất dễ bị nghẽn vật lý và khó đẩy xung nhịp cao vì dây dẫn chéo quá dài), M Core chia Back-end thành 2 cụm (Cluster) giống hệt nhau.

Mỗi Cụm (Trái và Phải) đều có đầy đủ đồ chơi:

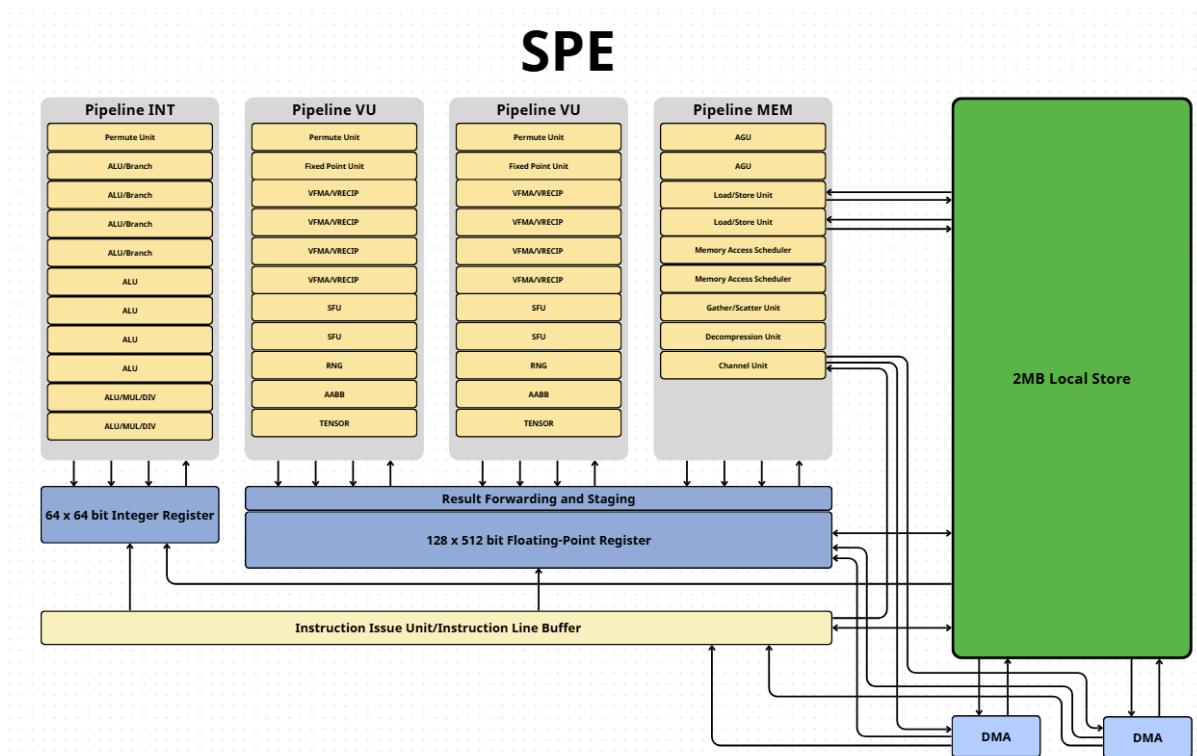
- **Khối Số nguyên (INT Schedulers):**
 - Mỗi bên có 3 cụm lập lịch (**32 INT, 24 INT, 24 INT**) nuôi 4 ALU.
 - *Tổng cộng cả nhân:* Có tới **8 ALU số nguyên** (bao gồm 4 ALU Branch). Xử lý logic và vòng lặp cực kỳ khủng khiếp.
- **Khối Số thực/Vector (FP Schedulers - Tập trung):**
 - Mỗi bên dùng một cụm lập lịch siêu lớn **96-entry FP Scheduler** thay vì chia nhỏ.
 - Mỗi cụm nuôi 2 cổng VALU/VMUL/FP ADD và 2 bộ FP MUL/DIV.
 - *Tổng cộng cả nhân:* 4 cổng Vector hỗn hợp, một sức mạnh tính toán khoa học đáng nể, được cách ly để luồng trái và luồng phải không tranh giành tài nguyên của nhau.

4. Memory Subsystem: Nút giao thông khổng lồ

Do phần trên bị chèn làm đôi, mọi dữ liệu cuối cùng sẽ đổ về một "nút thắt" chung ở dưới cùng. Nếu nút thắt này không đủ rộng, toàn bộ nhân sẽ bị tắc nghẽn.

- **10 Cổng AGU & MEM Schedulers:**
 - Mỗi cụm có một bộ **32 MEM Scheduler** nuôi 5 cổng AGU.
 - *Tổng cộng:* CPU này có thể tạo ra địa chỉ cho **10 thao tác bộ nhớ** (4 Store Data, 2 AGU Store Data, 4 Load Data) trong một chu kỳ nhịp đồng hồ! Một con số điên rồ.
- **Bộ đệm chờ (Load/Store Buffer):**
 - Để gánh 10 cổng AGU, bộ đệm Load siêu bự được trang bị: **212 Load Buffer** (lớn nhất từ đầu đến giờ) và 76 Store Buffer.
- **L1 D-Cache (128KB - 8-way):**
 - Kho L1D chung cho cả 2 cụm buộc phải trở ra đến **8 cửa giao tiếp** (mỗi cửa 32 B/Cycle) để phục vụ kịp thời lượng dữ liệu khổng lồ đẩy xuống từ 2 bên.
- **L2 Cache (4MB - 16-way):**
 - Một khối L2 độc lập cỡ trung bình (4MB) với liên kết 64 B/Cycle rộng rãi ra bên ngoài L3/RAM.

SPE



1. Hệ thống Bộ nhớ: Tạm biệt Cache, xin chào "Local Store" và "DMA"

Đây là điểm khác biệt cốt lõi nhất của SPE so với một CPU thông thường. Nó không dùng hệ thống L1/L2 Cache tự động.

- **2MB Local Store (Kho lưu trữ cục bộ):**
 - Thay vì dùng Cache (nơi phần cứng tự động tìm và nạp dữ liệu từ RAM một cách vô hình), SPE dùng một vùng nhớ SRAM 2 Megabyte gọi là Local Store.
 - *Đặc điểm:* Lập trình viên (hoặc Compiler) phải **tự tay quản lý** 2MB này. Bạn phải viết code chỉ định rõ: đem dữ liệu nào vào đây, cất ở tọa độ nào, tính toán xong thì đẩy ra đâu. Nó biến SPE thành một hộp cát (sandbox) tính toán độc lập và có độ trễ cực kỳ ổn định (predictable latency).
- **2 khối DMA (Direct Memory Access):**
 - Đây là những "chiếc xe tải" chờ hàng. Khi các đường ống tính toán (Pipeline) đang bận rộn xử lý khối dữ liệu A trong Local Store, DMA sẽ âm thầm chạy ra ngoài RAM hệ thống để cống khối dữ liệu B mang vào, và đồng thời chờ kết quả của khối dữ liệu trước đó trả về RAM. Quá trình này gọi là *Double-Buffering*, giúp SPE không bao giờ bị rảnh rỗi.
- **Instruction Issue Unit / Instruction Line Buffer:**
 - Khối nạp lệnh vô cùng đơn giản. Nó chỉ việc lấy lệnh đã được nạp sẵn trong Local Store và đẩy thẳng lên các thanh ghi mà không cần đoán trước hay xáo trộn gì cả.

2. Hệ thống Thanh ghi (Registers): Không gian khổng lồ cho Vector

Bởi vì không có bộ đối tên thanh ghi vật lý (PRF), sức mạnh của SPE phụ thuộc hoàn toàn vào số lượng thanh ghi kiến trúc mà nó cung cấp cho phần mềm.

- **64 x 64-bit Integer Register:** * Một khối thanh ghi số nguyên nhỏ gọn. Chủ yếu dùng để chạy các vòng lặp (loop counters), tính toán địa chỉ bộ nhớ, và các phép logic điều khiển cơ bản.
- **128 x 512-bit Floating-Point Register:**
 - Đây là con quái vật thực sự. Nó có tới 128 thanh ghi, và mỗi thanh ghi rộng tới **512-bit** (tương đương chuẩn AVX-512 của Intel).
 - Nó cho phép nhồi hàng chục con số thập phân (FP32 hoặc FP16) vào một thanh ghi để xử lý cùng một lúc (SIMD - Single Instruction Multiple Data).
- **Result Forwarding and Staging:** Mạng lưới dây dẫn giúp truyền ngay kết quả vừa tính xong ở chu kỳ này sang làm đầu vào cho phép tính ở chu kỳ tiếp theo mà không cần cất vào thanh ghi, giúp giảm độ trễ.

3. Các đường ống Thực thi (Pipelines): Cơ bắp đồ sộ

Toàn bộ sức mạnh của SPE được chia làm 4 đường ống khổng lồ hoạt động song song.

- **Pipeline INT (Đường ống Số nguyên):**
 - Giải quyết các phép toán logic và điều khiển luồng. Bao gồm 4 ALU/Branch (xử lý rẽ nhánh), 4 ALU thuần, và 2 ALU/MUL/DIV (Nhân chia số nguyên).
 - Có thêm khối **Permute Unit** để hoán đổi vị trí các byte dữ liệu.
- **2x Pipeline VU (Đường ống Vector - Trái tim của SPE):**
 - Mỗi SPE có tới 2 đường ống Vector giống hệt nhau, trang bị tận răng cho đồ họa và AI:
 - **4x VFMA / VRECIP:** Fused Multiply-Add (nhân rồi cộng ngay lập tức - cực kỳ quan trọng trong AI/Đồ họa) và Reciprocal (tính nghịch đảo, giúp chia siêu nhanh).

- **2x SFU (Special Function Unit):** Xử lý các hàm toán học phức tạp như Sin, Cos, Logarit, Căn bậc hai.
- **RNG (Random Number Generator):** Tạo số ngẫu nhiên bằng phần cứng, rất cần cho các thuật toán mô phỏng hoặc render (Monte Carlo).
- **AABB (Axis-Aligned Bounding Box):** Khối phần cứng **chuyên biệt cho Raytracing**. Nó giúp tính toán cực nhanh xem một tia sáng có va chạm với vật thể 3D nào trong không gian hay không.
- **TENSOR:** Khối chuyên nhân ma trận (Matrix Multiplication), sinh ra để tăng tốc các mô hình AI (như mạng nơ-ron học sâu - Deep Learning).
- **Pipeline MEM (Đường ống Bộ nhớ):**
 - Chuyên giao tiếp với 2MB Local Store. Gồm 2 AGU tính toán địa chỉ và 2 Load/Store Unit.
 - **Gather/Scatter Unit:** Vô cùng quan trọng cho xử lý Vector. Nó có thể nhặt các mảnh dữ liệu nằm rải rác khắp nơi trong bộ nhớ gom lại thành một cục 512-bit để đưa lên thanh ghi (Gather), và ngược lại (Scatter).
 - **Decompression Unit:** Tích hợp bộ giải nén ngay trên đường ống phần cứng. Rất hữu ích để bung nén texture (vân bề mặt đồ họa) hoặc tệp trọng số AI trực tiếp khi nạp từ Local Store, tiết kiệm băng thông.